

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	N Goutham	Reg. No.:	192472024
Section / Batch:	CSE-AI	Date:	20-07-2026

Code of Conduct

I N Goutham/192472024(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Goutham

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic		Focus	Q Nos
Information Extraction		Regex-Based Information Extraction	1
Pattern Matching		Regex Pattern Matching	2
Regular Expression Patterns		Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Section 1: Resume Information Extraction System

Aim

To develop a Python program using Regular Expressions to automatically extract candidate details from a resume, generate a structured profile summary, and determine candidate eligibility based on technical skills and work experience.

Problem Understanding

A multinational recruitment company receives thousands of resumes in text format. Manually reviewing each resume is time-consuming and prone to errors. The system should automatically extract the candidate's name, email address, mobile number, technical skills, and years of experience. It should also determine whether the candidate is eligible by checking if the candidate has Python skill and at least 2 years of experience.

Concept Used

- Python re (Regular Expressions) module
- re.search() – Finds the first matching pattern.
- re.findall() – Finds all occurrences of a pattern.
- Regular Expression pattern matching.
- String processing.
- Conditional statements (if-else).
- List operations.

Algorithm

1. Import the re module.
2. Store the resume information as a text string.
3. Extract the candidate's name using a regular expression.
4. Extract all email addresses.
5. Extract the mobile number.
6. Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
7. Extract the years of experience.
8. Display the extracted candidate information.
9. Check whether the candidate has at least 2 years of experience and possesses Python skill.
10. Display the candidate's eligibility status.

Sample Input:

```
import re
resume = """
Name: Natukula Goutham
Email: gouthamnatukula777@gmail.com
Mobile: 9876543211
Skills: Python, SQL, Machine Learning, NLP, Java
Experience: 3 years
```

"""

```
# Extract Name
name = re.search(r"Name:\s*(.*)", resume).group(1) if name else "Not Found"

# Extract Email
emails = re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)

# Extract Mobile Number
mobile = re.findall(r"\b[6-9]\d{9}\b", resume)

# Detect Skills
skill_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills = []
for skill in skill_list:
    if re.search(skill, resume, re.IGNORECASE):
        skills.append(skill)

# Extract Experience
exp = re.search(r"(\d+)\s+years?", resume, re.IGNORECASE).group(1) if exp else 0

# Display Summary
print("--- Candidate Summary ---")
print("Name:", name)
print("Email:", emails)
print("Mobile:", mobile)
print("Skills:", skills)
print("Experience:", experience, "Years")

# Eligibility
print("\n--- Eligibility ---")
if experience >= 2 and "Python" in skills:
    print("Candidate is Eligible")
else:
    print("Candidate is Not Eligible")
```

Sample Output

```
--- Candidate Summary ---
Name: Natukula Goutham
Email: ['gouthamnatukula777@gmail.com']
Mobile: ['9876543211']
Skills: ['Python', 'Java', 'SQL', 'Machine Learning', 'NLP']
Experience: 3 Years

--- Eligibility ---
Candidate is Eligible
```

```

import re

resume = """
Name: Natukula Goutham
Email: goutham.ai2027@gmail.com
Mobile: 9876543211
Skills: Python, SQL, Machine Learning, NLP, Java
Experience: 3 years
"""

# Extract Name
name = re.search(r"Name:\s*(.*)", resume)
name = name.group(1) if name else "Not Found"

# Extract Email
emails = re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)

# Extract Mobile Number
mobile = re.findall(r"\b[6-9]\d{9}\b", resume)

# Detect Skills
skill_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
skills = []

for skill in skill_list:
    if re.search(skill, resume, re.IGNORECASE):
        skills.append(skill)

# Extract Experience
exp = re.search(r"(\d+)\s+years?", resume, re.IGNORECASE)
experience = int(exp.group(1)) if exp else 0

# Display Summary
print("----- Candidate Summary -----")
print("Name:", name)
print("Email:", emails)
print("Mobile:", mobile)
print("Skills:", skills)
print("Experience:", experience, "Years")

# Eligibility
print("\n----- Eligibility -----")
if experience >= 2 and "Python" in skills:
    print("Candidate is Eligible")
else:
    print("Candidate is Not Eligible")

>>>
=====
----- Candidate Summary -----
Name: Natukula Goutham
Email: ['goutham.ai2027@gmail.com']
Mobile: ['9876543211']
Skills: ['Python', 'Java', 'SQL', 'Machine Learning', 'NLP']
Experience: 3 Years

----- Eligibility -----
Candidate is Eligible
>>>

```

Result / Conclusion

The Python program successfully extracts the candidate's name, email address, mobile number, technical skills, and years of experience using Regular Expressions. It generates a structured candidate profile and correctly determines whether the candidate satisfies the eligibility criteria of having Python skill and a minimum of 2 years of experience. The system helps automate the resume screening process, making recruitment faster and more efficient.

Section 2: Product Search System

Aim

To develop a Python program using Regular Expressions to search products based on exact keywords, prefixes, suffixes, partial keywords, and case-insensitive matching, and generate a report showing the matching products.

Problem Understanding

An e-commerce company wants to improve its product search system by allowing customers to search products using different keyword matching techniques. The system should perform exact, prefix, suffix, partial, and case-insensitive searches, display all matching products, and generate a report showing the total number of matching products for each search.

Concept Used

- Python re (Regular Expressions) module
- re.search() – Finds the first occurrence of a pattern.
- re.fullmatch() – Matches the complete string.
- Regular Expression pattern matching.
- Loops (for).
- Conditional statements (if).
- List operations.

Algorithm

1. Import the **re** module.
2. Create a list of product names.
3. Accept different search keywords.
4. Perform an exact keyword search.
5. Perform a prefix-based search.
6. Perform a suffix-based search.
7. Perform a partial keyword search.
8. Perform a case-insensitive search.
9. Display all matching products.
10. Display the total number of matching products for each search.

Sample Input

```
import re
products = [
    "Laptop",
    "Laptop Bag",
```

```

"Wireless Mouse",
"Gaming Mouse",
"Mechanical Keyboard",
"USB Cable",
"Bluetooth Speaker",
"Smart Watch",
"Laptop Stand",
"Wireless Earbuds"
]
exact_keyword = "Laptop"prefix_keyword = "Wire"suffix_keyword = "Bag"partial_keyword =
"Mouse"case_keyword = "smart"
# Exact Searchexact = [p for p in products if re.fullmatch(exact_keyword, p)]
# Prefix Searchprefix = [p for p in products if re.search(r"^" + prefix_keyword, p)]
# Suffix Searchsuffix = [p for p in products if re.search(suffix_keyword + r"$", p)]
# Partial Searchpartial = [p for p in products if re.search(partial_keyword, p)]
# Case-Insensitive Searchcase = [p for p in products if re.search(case_keyword, p,
re.IGNORECASE)]
print("---- Product Search Report ----")
print("\nExact Search:")print(exact)print("Total:", len(exact))
print("\nPrefix Search:")print(prefix)print("Total:", len(prefix))
print("\nSuffix Search:")print(suffix)print("Total:", len(suffix))
print("\nPartial Search:")print(partial)print("Total:", len(partial))
print("\nCase-Insensitive Search:")print(case)print("Total:", len(case))

```

Sample Output

---- Product Search Report ----

Exact Search:

['Laptop']

Total: 1

Prefix Search:

['Wireless Mouse', 'Wireless Earbuds']

Total: 2

Suffix Search:

['Laptop Bag']

Total: 1

Partial Search:

['Wireless Mouse', 'Gaming Mouse']

Total: 2

Case-Insensitive Search:

['Smart Watch']

Total: 1

```
import re

products = [
    "Laptop",
    "Laptop Bag",
    "Wireless Mouse",
    "Gaming Mouse",
    "Mechanical Keyboard",
    "USB Cable",
    "Bluetooth Speaker",
    "Smart Watch",
    "Laptop Stand",
    "Wireless Earbuds"
]

exact_keyword = "Laptop"
prefix_keyword = "Wire"
suffix_keyword = "Bag"
partial_keyword = "Mouse"
case_keyword = "smart"

# Exact Search
exact = [p for p in products if re.fullmatch(exact_keyword, p)]

# Prefix Search
prefix = [p for p in products if re.search(r"^" + prefix_keyword, p)]

# Suffix Search
suffix = [p for p in products if re.search(suffix_keyword + r"$", p)]

# Partial Search
partial = [p for p in products if re.search(partial_keyword, p)]

# Case-Insensitive Search
case = [p for p in products if re.search(case_keyword, p, re.IGNORECASE)]

print("----- Product Search Report -----")

print("\nExact Search:")
print(exact)
print("Total:", len(exact))

print("\nPrefix Search:")
print(prefix)
print("Total:", len(prefix))

print("\nSuffix Search:")
print(suffix)
print("Total:", len(suffix))

print("\nPartial Search:")
print(partial)
print("Total:", len(partial))

print("\nCase-Insensitive Search:")
print(case)
print("Total:", len(case))
|
```



```

>>>
=====
----- Product Search Report -----

Exact Search:
['Laptop']
Total: 1

Prefix Search:
['Wireless Mouse', 'Wireless Earbuds']
Total: 2

Suffix Search:
['Laptop Bag']
Total: 1

Partial Search:
['Wireless Mouse', 'Gaming Mouse']
Total: 2

Case-Insensitive Search:
['Smart Watch']
Total: 1
>>>

```

Result / Conclusion

The Python program successfully performs exact, prefix, suffix, partial, and case-insensitive product searches using Regular Expressions. It displays all matching product names and generates a report showing the total number of matches for each search type. This approach improves product searching in e-commerce applications by providing flexible and efficient keyword matching.

Section 3: University Registration System

Aim

To develop a Python program using Regular Expressions to validate student registration details such as register number, institutional email, course code, semester, and mobile number, and generate a final registration status report.

Problem Understanding

A university is developing an online registration portal to verify student information before course enrollment. The system should validate the student's register number, institutional email address, course code, semester, and mobile number using Regular Expressions. It should display appropriate validation messages for each field and generate a final registration status indicating whether the registration is successful.

Concept Used

- Python re (Regular Expressions) module
- re.fullmatch() – Matches the complete pattern.
- Regular Expression pattern matching.
- Conditional statements (if-else).
- Boolean variables.
- String validation.

Algorithm

1. Import the **re** module.
2. Read the student's register number, email, course code, semester, and mobile number.
3. Validate the register number using a regular expression.
4. Validate the institutional email address.
5. Validate the course code.
6. Validate the semester.
7. Validate the mobile number.
8. Display validation messages for each field.
9. Check whether all fields are valid.
10. Display the final registration status.

Sample Input

```
import re
# Student Details
reg_no = "23AIA10234"
email = "goutham.23ai@saveetha.com"
course_code = "DSA03"
semester = "5"
mobile = "9876543211"
valid = True
# Register Number Validation
if re.fullmatch(r"\d{2}[A-Z]{3}\d{5}", reg_no):
    print("Register Number: Valid")
else:
    print("Register Number: Invalid")
    valid = False
# Email Validation
if re.fullmatch(r"[A-Za-z0-9._%+-]+@saveetha\.com", email):
    print("Email: Valid")
else:
    print("Email: Invalid")
    valid = False
# Course Code Validation
if re.fullmatch(r"[A-Z]{3}\d{2}", course_code):
    print("Course Code: Valid")
else:
    print("Course Code: Invalid")
    valid = False
# Semester Validation
if re.fullmatch(r"[1-8]", semester):
    print("Semester: Valid")
else:
    print("Semester: Invalid")
    valid = False
# Mobile Number Validation
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number: Valid")
else:
    print("Mobile Number: Invalid")
    valid = False
# Final Status
print("\n--- Registration Status ---")
if valid:
    print("Registration Successful")
else:
    print("Registration Failed")
```

Sample Output

```
Register Number: Valid
Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid
```

```
--- Registration Status ---
Registration Successful
```

```
import re

# Student Details
reg_no = "23AIA10234"
email = "goutham.23ai@saveetha.com"
course_code = "DSA03"
semester = "5"
mobile = "9876543211"

valid = True

# Register Number Validation
if re.fullmatch(r"\d{2}[A-Z]{3}\d{5}", reg_no):
    print("Register Number: Valid")
else:
    print("Register Number: Invalid")
    valid = False

# Email Validation
if re.fullmatch(r"[A-Za-z0-9._%+-]@saveetha\.com", email):
    print("Email: Valid")
else:
    print("Email: Invalid")
    valid = False

# Course Code Validation
if re.fullmatch(r"[A-Z]{3}\d{2}", course_code):
    print("Course Code: Valid")
else:
    print("Course Code: Invalid")
    valid = False

# Semester Validation
if re.fullmatch(r"[1-8]", semester):
    print("Semester: Valid")
else:
    print("Semester: Invalid")
    valid = False

# Mobile Number Validation
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number: Valid")
else:
    print("Mobile Number: Invalid")
    valid = False

# Final Status
print("\n----- Registration Status -----")
if valid:
    print("Registration Successful")
else:
    print("Registration Failed")

Register Number: Valid
Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid

----- Registration Status -----
Registration Successful
```

Result / Conclusion

The Python program successfully validates the student register number, institutional email address, course code, semester, and mobile number using Regular Expressions. It displays validation messages for each field and generates the final registration status. The system ensures that only students with valid information can complete the registration process, improving the accuracy and reliability of the university's online registration portal.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
-------------------	----------------------------------	-------------------------	--------------------------	----------------------	---------	------------	---------

1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____